International Islamic University Chittagong

Department of Computer Science and Engineering

B. Sc. in CSE Midterm Examination, Autumn 2022

Course Code: CSE 2421 Course Title: Computer Algorithms

Total marks: 30 *Time: 90 minutes*

[Answer all the questions; in some questions, there might be options; Figures in the right hand margin indicate full marks.]

		CO	DL	
1.				
a)	Let **K = ((X MOD 2)+1)*2**, where **X** is the last digit of your ID. Now, find the asymptotic upper bound of the following recurrence by using master method. **T(n) = KT(n/2) + n** Verify your answer through recursion tree method.	CO4	N	4
b)	Compare the time complexity of the following two functions. Which one runs faster.	CO4	E	3

```
void Chip (int n)
{
  for (int i = 1; i <= n; i++)
    for (int j = 1; j <= i; j = j*2)
      k = k + 1;
}

void Dale (int n)
{
  for (int i = 1; i <= n; i = i*2)
    for (int j = 1; j <= i; j++)
      k = k + 1;
}
```

c)	Show that the asymptotic upper bound of the following function is not $O(n^2)$: $An^3 + Bn^2 + C$. Assume that A, B, and C are any constants.	CO4	N	3
	OR			
	Show that the asymptotic lower bound of the following function is not $\Omega(n^2)$: $An^3 + Bn^4 + Cn^5$. Assume that A, B, and C are any constants.			
2.				
a)	Prove that we can convert an array into a heap in **O(n)** time.	CO4	N	4
b)	Write down the recurrence equation for the running time of quicksort algorithm in which partition always produces a 9-to-4 proportional split. Solve the recurrence using recursion tree and show that in this case running time is **O(nlgn)**.	CO3	N	3

c) The operation **Heap-Delete(A, i)** deletes the item in node **i** from heap **A**. Give an implementation of **Heap-Delete** that runs in **O(lgn)** time for an n-element max-heap. CO2 A 3

OR

Show the steps of the operation **Extract-Max()** on the following Max-Heap $A = \langle 10, 9, 8, 7, 6, 5, 4, 3, 2, 1 \rangle$.

3.

a) Find an optimal parenthesization of a matrix-chain product whose sequence of dimensions is $\langle 7, 4, 5, 3, 3 \rangle$. CO1 A 4

OR

Find a Longest Common Sequence of the following two sequences using dynamic programming and show the steps.
$X = \langle A, B, C, D, E, F \rangle$
$Y = \langle D, C, B, E, A, F, E \rangle$

b) Give a brief argument why we cannot apply the technique of dynamic programming if the problem does not have optimal substructure property. CO5 N 2

c) Show the operation of the first PARTITION of the quick sort algorithm on the array CO1 A 2

$A = \langle 6, 4, 8, 9, 2, 3, 7, 4, 8, 5 \rangle$

d) What is the smallest value of **n** such that an algorithm whose running time is $100*(X+1)*n^2$ runs faster than an algorithm whose running time is 2^n on the same machine? Assume that **X** is the last digit of your ID. CO4 N 2

(Autumn-22)

1(a)

Here, $k = ((x \bmod 2) + 1) * 2$.

$= ((7 \bmod 2) + 1) * 2$

$= 4$

So, our given equation will be converted to

$T(n) = 4T(n/2) + n$ ..... (i)

Our master theorem's equ. is,

$T(n) = aT(n/b) + O(n^k \log^p n)$ ..... (ii)

So, comparing (i) and (ii) $\Rightarrow$

$a = 4, b = 2, k = 1, p = 0$

Here, $\log_b a = \log_2 4 = 2$ So, $\log_b a > k$

Now, $T(n) = O(n^{\log_b a})$

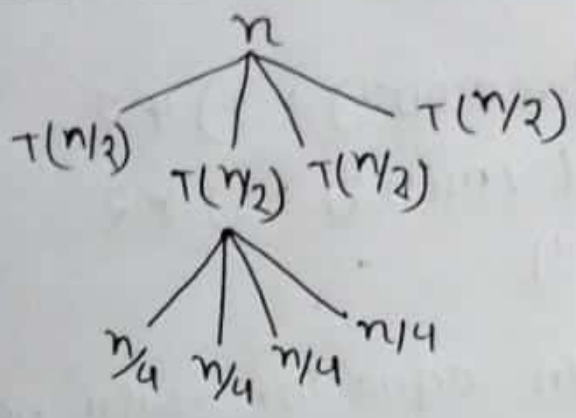
or, $T(n) = O(n^{\log_2 4})$ [$\because \log_b a > k$]

$\therefore T(n) = O(n^2)$

Now, verifying the my answer using recursion tree method.

Our equation is,

$T(n) = 4T(n/2) + n$

level	node	Recursion tree	level sum
0	4^0		$4^0(n) = n$
1	4		$4^1(n/2) = 2n$
2	4^2		$4^2(n/4) = 4n$
⋮	⋮	⋮	⋮
i	4^i	$n/2^i$	$4^i(n/2^i) = 2^i n$
n	4^n	T(1) T(1) T(1) T(1)	4^n

So, $T(n) = 4^h + (1) + \sum_{i=0}^{h-1} 2^i n$ ---(iii)

because base case is 1 so,

$$\frac{n}{2^h} = 1$$

$$\Rightarrow n = 2^h$$

$$\Rightarrow \log_2 n = \log_2 2^h$$

$$\therefore h = \log_2 n \quad \text{---(iv)}$$

Putting h's values in (iii) $\Rightarrow$

$$T(n) = 4^{\log_2 n}\, T(1)\, 0 + \sum_{i=0}^{h-1} 2^i n$$

$$= n^{\log_2 2^2}\, T(1) + \sum_{i=0}^{h-1} 2^i n$$

$$= n^2 T(1) + n[1 + 2^1 + 2^2 + \cdots + 2^{h-1}]$$

$$= n^2 T(1) + n\, 2^h - 1$$

$= n^2 T(1) + n(2\log_2 n - 1)$

$T(n) = n^2 T(1) + n(n^{\log_2 2} - 1)$

$= n^2 T(1) + n(n-1)$

$\therefore T(n) = \Theta(n^2)$ (Proved)

1(b)

Let's analyse the codes,

#1
```
void chip (int n)
{
  for(int i=1; i<=n; i++) ——— n
  for(int j=1: j<=i; j=j*2) — nxlogn
   k=k+1 ——— nxlogn
}
```

Because i's value will reach n which will definitely take $O(n)$ time,
So, for #1 $O(n \log n)$

#2
```
void Date (int n) {
  for(int i=1; i<=n; i=i*2) — logn
  for(int j=1; j<=1; j++) ——— lognxn
   k=k+1; ——— lognxn
}
```

So, the time complexity of #2 is $O(n \log n)$.

1(c)

Suppose, $An^3 + Bn^2 + C$ is $O(n^2)$

So, there will exist an constant k and n_0 such that $n \geq n_0$

$$An^3 + Bn^2 + C \leq kn^2$$

or, $An + B + \frac{C}{n^2} \leq k$

So, now for any value n the dominent part will always be the left side of equation. It totally contradicts our assumption.

So, $An^3 + Bn^2 + C$ is not upper bounded by $O(n^2)$

1(c) Or

Suppose, $An^3 + Bn^4 + Cn^5$ is $\Omega(n^2)$

So, the will exist a positive constant k such that for all $n \geq n_0$

$$kn^2 \leq An^3 + Bn^4 + Cn^5$$

Or, $k \leq An + Bn^2 + Cn^3$

for any value n the function will hold the condition we assumed.

$\therefore An^3 + Bn^4 + Cn^5$ is bounded by $\Omega(n^2)$.

(Ans)

Ans. to the Q. no - 02(a)

Yes, we can convert an array into a heap in O(n) time using heapify method.

Heapify is the process of creating heap data structure from a binary tree represented using an array. We start from the first index which is a non-leaf node means doesn't have any child node.

1. Start from the last non-leaf node (index $n/2 - 1$) to the first element (index 0) of the array.

2. Perform a "sift Down" operation. Compare the element with its children and swap it (depending on whether it's a max-heap or min-heap) with child if necessary. Continue this process recursively until the heap property is satisfied.

Since the height of a heap is logarithmic ($O(\log n)$), the number of comparisons and swaps needed for each element to reach its correct position is bounded by the

height of the heap. When considering the entire sequence of heapify operations, the total work done is proportional to the number of elements in the array that is O(n).

Therefore, we can convert an array into heap in O(n) time complexity.

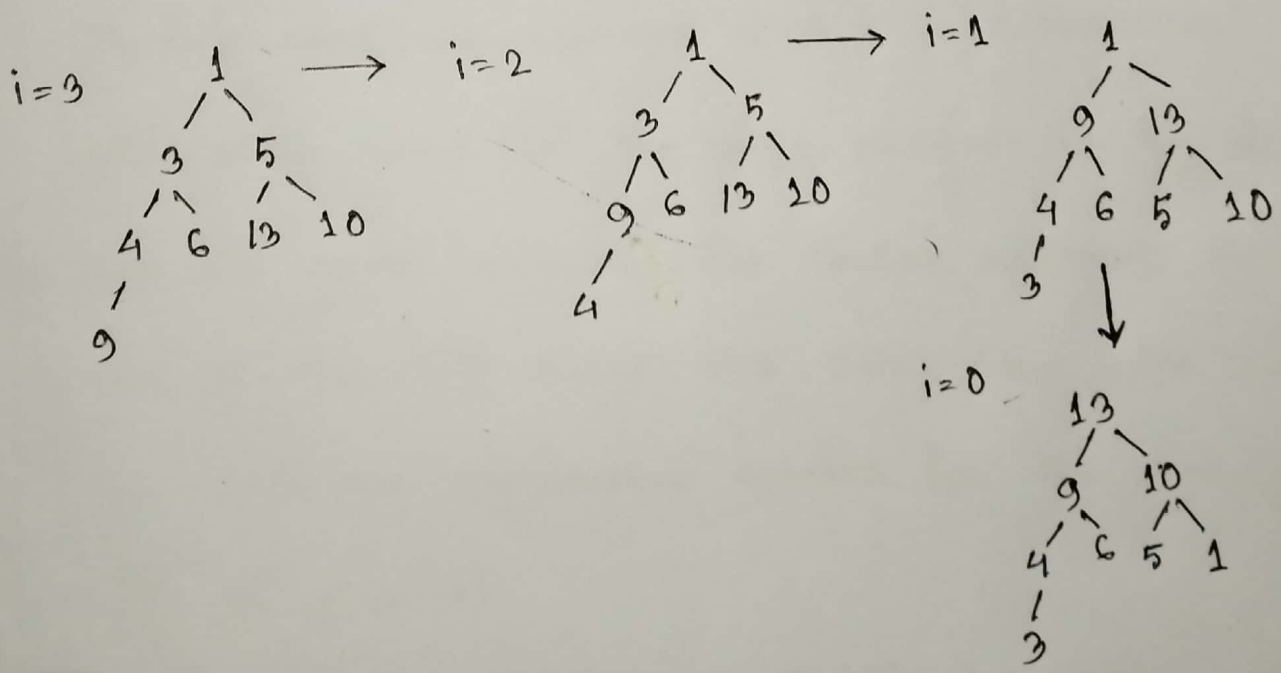
Now, we are going to build a max-heap.

0	1	2	3	4	5	6	7
1	3	5	4	6	13	10	9

node, n = 8

non-leaf nodes are = 0 to $\lfloor 8/2 \rfloor - 1$

= 0 - 3



Ans. to the Q.no - 02 (b)

Quicksort is a popular sorting algorithm that has an average case time complexity of $O(n \log n)$. However, in the worst case, the time complexity can be as high as $O(n^2)$. One way to improve the worst case behaviour of quicksort is to use a proportional split when partitioning the array. This means that the partition divides the array into two subarrays whose sizes are in a fixed proportion to each other.

In this case, we consider a 9 to 4 proportional split, which means that the large subarray is $9/13$ of the total array size and the smaller subarray is $4/13$ of the total array size. Using this split we can write the recurrence equation for the running time of quicksort.

$$T(n) = T(9n/13) + T(4n/13) + O(n)$$

Here, $T(n)$ represents the running time of quicksort on an input of size n. The first term, $T(9n/13)$, represents the time taken to sort the larger partition ($9n/13$ elements), and the second term, $T(4n/13)$, represents the time taken to sort the smaller partition ($4n/13$ elements). The last term, $O(n)$, corresponds to the time taken for the partitioning step.

To solve this recurrence equation, we can use a recursion tree, where each node represents the running time for a subarray of a certain size. We start with the root node, which represents the running time for the entire array of size n. At each level of the tree, we split the current subarray into two subarrays according to the 9 to 4 proportional split and create two children nodes. We continue this process until we reach subarrays of size 1, which represents the best case. Using the recursion tree we can show that the total number of nodes in the tree is $9n$. This

is because each level of the tree reduces the size of the subarrays by a factor of 13/9, and there are 9/4 times as many nodes at each level compared to the level above it. Each node require O(n) time to process, so that the total running time is $\theta(n \log n)$.

Therefore, by using a 9-to-4 proportional split, we can improve the worst case behavior of quick sort to $\theta(n \log n)$.

2(c)

Now, the implementation of heap-delete that runs in O(logn) time for n-element max-heap:

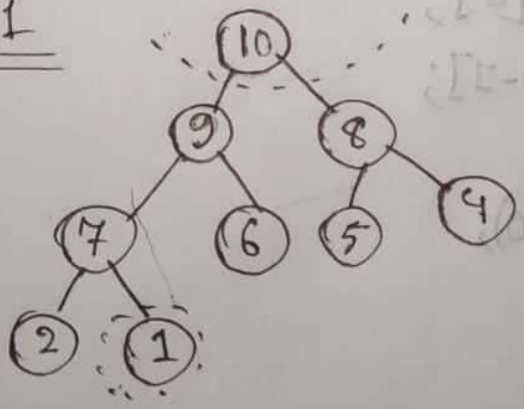
```
int heapDelete(vector<int>& heap, int& n)
{
 if(n==0)
 {
 cout<<"heap is empty!"<<endl;
 return -1;
 }
 int root=heap[0];
 heap[0]=heap[n-1];
 n--;
 heapify(heap,n,0);
 return root;
 }
int main()
{
 vector<int> heap={10,9,8,7,6,5,4,2,1}
 int n=heap.size();
 cout<<"max heap is:";
 for(int num:heap)
 cout<<num<<" "<<endl;
 int deletdelement=heapdelete(heap,n);
 cout<<"deleted elements is:"<<endl;
 cout<<"heap after deletion:";
 for(int i=0;i<n;i++)
 cout<<heap[i]<<" "<<endl;
```

2(c) on

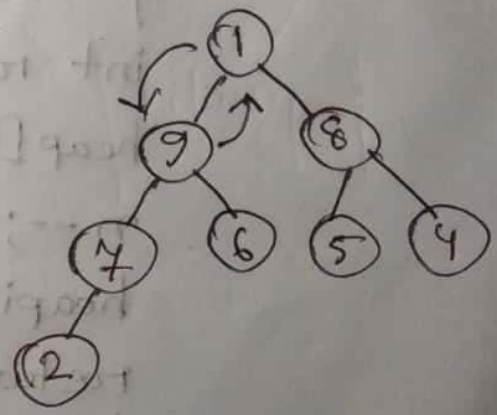
⇒ Given that Array is,

$A = \{10, 9, 8, 7, 6, 5, 4, 2, 1\}$

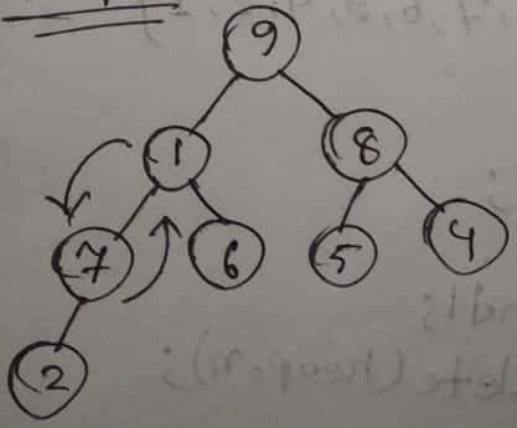
step-1



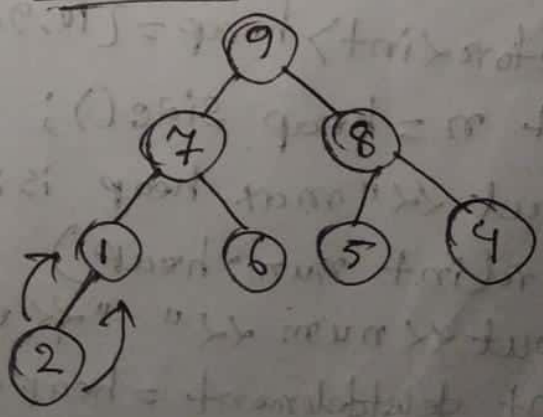
step-2



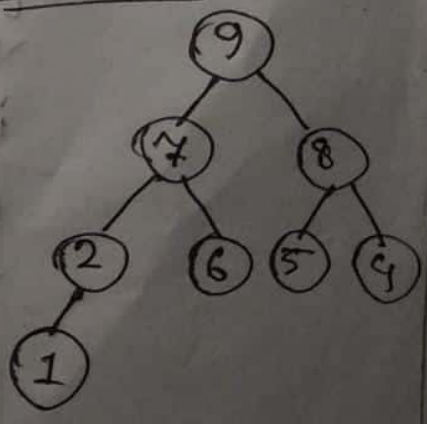
step-3



step-4



step-5



Ans. to the Ques. No. 3(a)

Given the sequence, $P = \{7, 4, 5, 3, 3\}$, Fill up following m and s tables,

matrix	A_1	A_2	A_3	A_4
dimension	7×4	4×5	5×3	3×3
	$P_0 \times P_1$	$P_1 \times P_2$	$P_2 \times P_3$	$P_3 \times P_4$

m

	1	2	3	4
1	0	140	144	180
2		0	60	96
3			0	45
4				0

s

	2	3	4
1	1	1	1
2		2	3
3			3

$m[1,2]$

When $k=1$,

$m[1,2] = m[1,1] + m[2,2] + 7 \times 4 \times 5$

$= 0 + 0 + 140 = 140$

$s[1,2] = 1 \quad [\because k = 1]$

$m[2,3]$

When $k=2$,

$m[2,3] = m[2,2] + m[3,3] + 4 \times 5 \times 3 = 0 + 0 + 60 = 60$

$s[2,3] = 2 \quad [\because k = 2]$

$m[3,4]$

When $k=3$,

$m[3,4] = m[3,3] + m[4,4] + 5\times3\times3$

$= 0 + 0 + 45 = 45$

$s[3,4] = 3$ $[\because k=3]$

$m[1,3]$

When $k=1$,

$m[1,3] = m[1,1] + m[2,3] + 7\times4\times3$

$= 0 + 60 + 84 = 144$

When $k=2$,

$m[1,3] = m[1,2] + m[3,3] + 7\times5\times3$

$= 140 + 0 + 105 = 245$

$\therefore m[1,3] = 144$ when $k=1$ and $s[1,3] = 1$ $[\because k=1]$

$m[2,4]$

When $k=2$,

$m[2,4] = m[2,2] + m[3,4] + 4\times5\times3$

$= 0 + 45 + 60$

$= 105$

When $k=3$,

$m[2,4] = m[2,3] + m[4,4] + 4\times3\times3$

$= 60 + 0 + 36 = 96$

$\therefore m[2,4] = 96$ when $k=3$ and

$s[2,4] = 3$ $[\because k=3]$

$m[1,4]$

When $k=1$,

$m[1,4] = m[1,1] + m[2,4] + 7\times4\times3$

$= 0 + 96 + 84 = 180$

When $k=2$,

$m[1,4] = m[1,2] + m[3,4] + 7\times5\times3$

$= 140 + 45 + 105$

$= 290$

When $k=3$,

$m[1,4] = m[1,3] + m[4,4] + 7\times3\times3$

$= 144 + 0 + 63 = 207$

$\therefore m[1,4] = 180$ when $k=1$ and

$s[1,4] = 1$.

Optimal parenthesization for the given matrices,
$(A_1)((A_2 \times A_3)(A_4))$.

Ans. to the Ques. No. 3(Or, a)

The given strings,

$X = \{A, B, C, D, E, F\}$

$Y = \{D, C, B, E, A, F, E\}$

LCS Table

		0	1	2	3	4	5	6	7
		y_j	(D)	C	B	(E)	A	(F)	E
0	x_i	0	0	0	0	0	0	0	0
1	A	0	0	0	0	0	↖1	←1	←1
2	B	0	0	0	↖1	←1	←1	←1	←1
3	C	0	0	↖1	←1	←1	←1	←1	←1
4	(D)	0	(↖1)	(←1)	(←1)	←1	←1	←1	←1
5	(E)	0	↑1	←1	←1	(↖2)	(←2)	←2	↖2
6	(F)	0	↑1	←1	←1	↑2	←2	(↖3)	←3

Hence, the log longest common ~~su~~ sequence of the given two sequences is DEF, which is of length 3.

3(b) Give a brief argument why we can't apply the technique of dynamic programming if the problem does not have optimal substructure property.

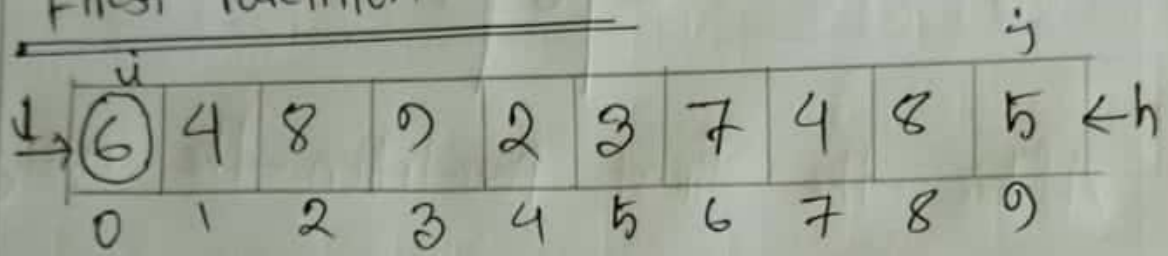
Ans: Dynamic programming is a technique for solving problems by breaking them down into smaller, overlapping subproblems and solving each subproblem once. However, this technique requires the problem to have a certain property called optimal substructure, which means that the optimal solution to the problem can be constructed from solutions to its subproblems. If a problem does not have this property, then dynamic programming cannot be used to solve it efficiently. In other words, without optimal substructure, dynamic programming may not work, and other methods will be needed to solve the problem.

3.c) Show the operation of the first PARTITION of the quick sort algorithm on the array -

$A = \{6, 4, 8, 9, 2, 3, 7, 4, 8, 5\}$

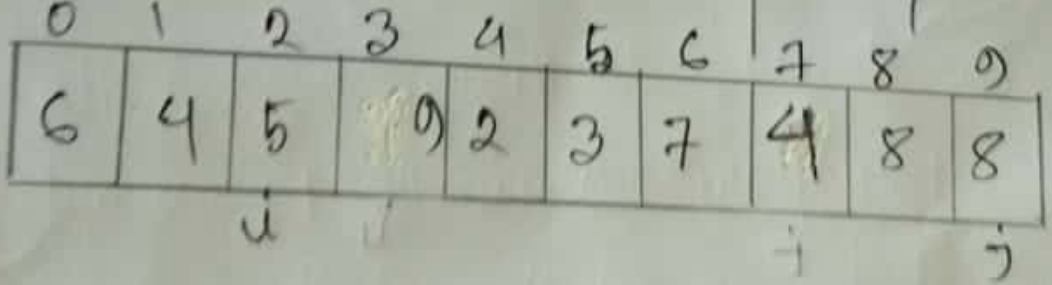
Ans: Array, $A = \{6, 4, 8, 9, 2, 3, 7, 4, 8, 5\}$

First Partition Step:



Here,
lower index = l
higher index = h
pivot = A[l] = 6

Condition,
A[i] ≤ pivot
{ i++;
}
A[j] > pivot
{ j--;
}
i < j
{ swap(A[i], A[j]);
}



0	1	2	3	4	5	6	7	8	9
6	4	5	4	2	3	7	9	8	8
			i				j		

0	1	2	3	4	5	6	7	8	9
6	4	5	4	2	3	7	9	8	8
					j	i			

0	1	2	3	4	5	6	7	8	9
3	4	5	4	2	6	7	9	8	8

After the first partition the array will look like,

$A = \{3, 4, 5, 4, 2, 6, 7, 9, 8; 8\}$

Ans. to the Ques. No. 3(d)

The expression according to my ID:

$100 * (3+1) * n^2 = 400\, n^2$

Now to find the solution we have to solve this inequality:

$$400\, n^2 < 2^n$$

Taking logarithm base 2 on both sides,

$$\log_2(400\, n^2) < \log_2 2^n$$

Using properties of logarithms,

$$\log_2 400 + \log_2 n^2 < n$$

or, $\log_2 400 + 2\log_2 n < n$

Substituting the value of $\log_2 400 \approx 8.644$,

$$8.644 + 2\log_2 n < n$$

or, $8.644 < n - 2\log_2 n$

Using a numerical solver, we find $n = 15.53$ (approximately). Since n must be a positive

integer, the smallest value of n that satisfies the inequality is $n=16$.

So, when n is 16 or greater, the algorithm with running time $100(n^2)$ is faster then the algorithm with running time 2^n.